Slot-C

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 1 – ANALYTICAL PROBLEM SOLVING

Course Code:	CSA1402	**Course Title:**	COMPILER DESIGN
CO Assessed:	CO1 – Imitation (L1)	**Assessment:**	Assessment Tool 1 – ANALYTICAL PROBLEM SOLVING
Weightage:	40% of CIA		
CO1 Statement:	Apply lexical analysis for token specification and recognition using LEX tool. (BL3)		
Student Name:	HARI CHARAN.P	**Reg. No.:**	192511107
Section / Batch:	CSE	**Date:**	16/6/2026

Code of Conduct

I Haricharan.P *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: Hari.Charan.P

Instructions

- Develop a modular and efficient Python program that satisfies all the functional requirements specified in the problem statement.
- Demonstrate the correctness of the implementation using suitable test cases and justify the output obtained.
- Follow good programming practices, including meaningful variable names, proper code indentation, comments, and error handling wherever applicable.
- Duration: 30 minutes.

Q. No	Problem Understanding (20)	Method Selection (20)	Solution Process (25)	Accuracy of Calculation (20)	Interpretation of Results (15)	Total Score (out of 100)
1	4 / 4	3 / 4	4 / 4	4 / 4	3 / 4	91.25
2	4 / 4	3 / 4	3 / 4	4 / 4	4 / 4	88.75
3	3 / 4	4 / 4	3 / 4	4 / 4	3 / 4	85
4	4 / 4	4 / 4	3 / 4	3 / 4	3 / 4	85
5	3 / 4	3 / 4	4 / 4	4 / 4	4 / 4	90
OVERALL RUBRICS VALUE						
	/ 4	/ 4	/ 4	/ 4	/ 4	88
	/ 20	/ 20	/ 25	/ 20	/ 15	/ 100

N. D

OR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Le D
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	P u r e
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretatio with mir omissions	

CO-1-AT-1

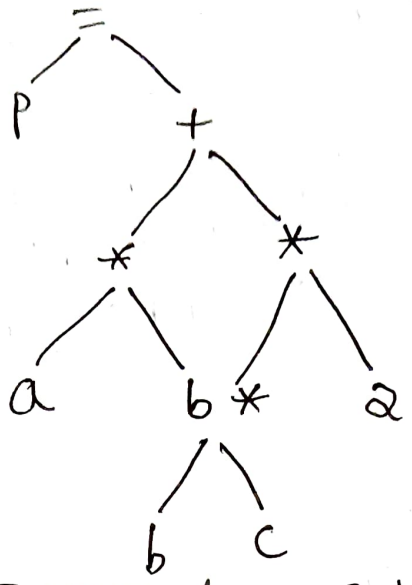
HARI CHARAN P
192511107
CSA-0379 - Compiler Design

Trace the processing of the input expression P = (a*b)+(b*c)*2 through all phases of a compiler

a) Identify and list all tokens generated during lexical analysis.

Token	type
P	Id_1
=	Assignment operator
(	Left Paranthesis
a	Id_2
*	Multiplication operator
b	Id_3
)	Right Paranthesis
+	Addition operator
(	Left Paranthesis
b	Id_4
*	Multiplication operator
c	Id_5
)	Right Paranthesis
*	Multiplication operator
2	Integer

) Construct syntax tree during Parsing Phase



c) Generate Intermediate Code for given expression

$t_1 = a * b$
$t_2 = b * c$
$t_3 = t_2 * 2$
$t_4 = t_1 + t_3$
$P = t_4$

d) Provide Suitable optimization for Intermed Code

$t_1 = a * b$
$t_2 = b * c$
$P = t_1 + (t_2 * 2)$

e) Significance of target code

- ★ Converts intermediate code into assembly
- ★ Optimized for the target processor.
- ★ Ready for excution by the CPU.

For each of the following regular expression, construct the language. Write any 5 valid strings for each regular expression

a) $(\varepsilon + aa')'$

Eg:-

☆ ε ☆ aa ☆ aaaa ☆ aaaaaa

☆ aaa aaaaaa

b) $(a+b^*)^* abb$

Eg:-

☆ abb ☆ aabb ☆ babb ☆ bbabb ☆ aaabb

c) $(a+b)^* aba (a+b)^*$

Eg:-

☆ aba ☆ aaba ☆ abab ☆ ababbb ☆ baabaa

d) $(a+ab)^*$

Eg:-

☆ a ☆ ab ☆ aa ☆ aab ☆ abab

e) $(aba)+(ab)$

Eg:-

☆ aba ☆ ab

3. i) Find the no. of tokens in following c statement is

```
main()
{
int a=10;
Char b="abc";
int c=30;
Char d="xyz";
in /*Comment*/ t m=40.5;
}
```

(A) Total tokens = 24

(ii) Identify the lexical errors in the following C statement is

```
main()
{
int a=10;      int x=10, 4=20;
Char b= "abc"; Char *a;
int            a=&x
               x=1xab;
               }
```

Errors:-

☆ a=&x :⇒ Type mismatch (a is char*, &x is int*).

☆ 1xab ⇒ Invalid numeric literal (lexical error).

4. Find a regular expression corresponding to each of the following subset of $\{0,1\}^*$.

(i) The language of all strings that have an even number of 0's.

(A) $1^*(01^*01^*)^*$

(ii) The language of all strings that contains atleast one 1.

(A) 0^*10^*

(iv) The language of all string in which both the number of 0's and the number of 1's are odd

(A) $(00(00)^*1(11)^*)+$

The language of all strings with 1 and end with 0.

1(0+1)*0

The language of all strings that do not contain consecutive 1's.

(0+10)*1?

Consider a lexical analyzer for a simplified programming language. The language consists of

int Var123 = 12345; float Var456 = 123.45;
if (Var123 == Var456) { return ; }

How many tokens are there in the input string?

18

Provide a list of tokens categorized by their types

Token	type
Var123	Id
=	Operator
12345	Integer
;	Seperator
float	keyword
Var456	Id
=	Operator
123.45	floating literal
;	Seperator
if	keyword
	Seperator

{ Separator
return keyword
; Separator
} Separator

Assuming a finite state machine (FSM) is used for lexical analysis, estimate the no. of state transition required to tokenize the given input string. Provide your reasoning based on transition needed for each token type

keyword -> 3-5 each
Identifiers => 6 each
Integer literal => 5
floating literal => 7
operators => 1-2 each
Separators => 1 each

Estimated Total State transitions ≥ 55-